

vtu-easy.com

DIGITAL SYSTEM DESIGN USING VERILOG (BEC302)

MODULE – 4 INTRODUCTION TO VERILOG

Topics Covered:

- Structure of Verilog Module • Operators • Data Types
- Styles of Description • Verilog Data Flow Description

Department of ECE
VTU B.E. – 3rd Semester
vtu-easy.com

TABLE OF CONTENTS

Click any topic to navigate directly to that section. vtu-easy.com

[1. Why HDL? – What is HDL & Need for HDL](#)

[2. A Brief History of Verilog](#)

[3. Structure of Verilog Module](#)

[3.1 Module Declaration & Syntax](#)

[3.2 Verilog Ports \(input / output / inout\)](#)

[3.3 Half Adder Example](#)

[4. Verilog Operators](#)

[4.1 Logical Operators \(Bitwise, Boolean, Reduction\)](#)

[4.2 Relational Operators](#)

[4.3 Arithmetic Operators](#)

[4.4 Shift and Rotate Operators](#)

[5. Verilog Data Types](#)

[5.1 Nets \(wire\)](#)

[5.2 Registers \(reg\)](#)

[5.3 Vectors](#)

[5.4 Integer, Real, Parameters, Array](#)

[6. Styles \(Types\) of Descriptions](#)

[6.1 Behavioral Description](#)

[6.2 Structural Description](#)

[6.3 Dataflow Description](#)

[6.4 Switch Level Description](#)

[6.5 Mixed-Type Description](#)

[6.6 Mixed-Language Description](#)

[7. Verilog Dataflow Description \(Detailed\)](#)

[7.1 Highlights of Dataflow Description](#)

[7.2 Concurrent vs Sequential Execution](#)

[7.3 Structure of Dataflow Description](#)

[7.4 Signal Declaration](#)

[7.5 Signal Assignment Statements](#)

[7.6 Continuous Assignment](#)

[7.7 Concurrent Assignment](#)

[7.8 Constant Declaration](#)

[7.9 Conditional Signal Assignment](#)

[7.10 Selected Signal Assignment](#)

[7.11 Delay Assignment](#)

[8. Quick Revision – All Topics at a Glance](#)

1. Why HDL?

1.1 What is Hardware Description Language (HDL)?

A Hardware Description Language (HDL) is a **computer-aided design (CAD) tool** used for the modern digital design, simulation, verification, and synthesis of digital systems. Unlike conventional software programming languages, HDL is specifically crafted to model hardware behaviour and structure at various levels of abstraction — from transistor level to system level.

HDL allows a designer to describe a hardware system in a text-based, human-readable format that closely resembles other high-level programming languages like C. This makes it easy to write, debug, simulate, and synthesize digital circuits without physically building them first.

Two most widely used Hardware Description Languages:

- VHDL (VHSIC Hardware Description Language) – more verbose, strongly typed
- Verilog HDL – simpler syntax, based on C language, widely used in industry

1.2 Need for HDL

The rapid advancement in semiconductor technology has dramatically increased the power, speed, and complexity of modern digital systems. Due to this explosive growth:

- Discrete ICs (AND, OR, NOT gates) are no longer capable of realising complex modern systems.
- High-density programmable chips such as ASICs (Application Specific Integrated Circuits) and FPGAs (Field Programmable Gate Arrays) are now used to implement complex digital systems on a single chip.
- Designing such systems manually is error-prone and extremely time-consuming.
- HDL enables designers to write, simulate, and verify designs before hardware fabrication, saving time and cost.
- HDL packages implement simulators and test benches, making debugging straightforward.
- Synthesis tools can automatically convert HDL code into a gate-level netlist for chip fabrication.

 **NOTE:** HDL is to hardware what C/C++ is to software — it bridges the gap between algorithm and silicon.

2. A Brief History of Verilog

Understanding the evolution of Verilog helps appreciate why it is the industry-standard HDL for digital design today.

1. In 1983, a company called Gateway Design Automation developed Verilog HDL as a hardware description language for its newly introduced logic simulator Verilog_XL. It was initially a proprietary language.
2. Gateway Design Automation was acquired by Cadence Design Systems in 1989. Cadence subsequently made Verilog available as a public domain language to encourage wider adoption.
3. The language gained enormous popularity due to its simplicity and C-like syntax.
4. In December 1995, Verilog HDL was officially standardised as IEEE Standard 1364-1995 — making it a vendor-independent, internationally recognised standard.
5. The language is currently maintained by the Open Verilog International (OVI) organization, which continues to evolve the standard.
6. Verilog code structure is based on the C software language, making it easy to learn for programmers.

Key Feature: Verilog is case-sensitive. Keywords are lowercase. "halfadder" and "Halfadder" are treated as two completely different module names.

3. Structure of Verilog Module

The **module** is the fundamental building block in Verilog. Every Verilog design is made up of one or more modules. A module can represent anything from a simple gate to a complete microprocessor.

A Verilog module has two main parts:

7. Declaration Section – defines the module name, input ports, output ports, internal signals, wires, registers, parameters, functions, and tasks.
8. Body / Statement Section – describes the functional behaviour or structural connections of the module using assignment statements, always blocks, initial blocks, or module instantiations.

3.1 Module Syntax

General Syntax of a Verilog Module:

```
module <name_of_module> (port_list);
// — Declaration Section —
input  <port_names>;           // input ports
output <port_names>;           // output ports
inout  <port_names>;           // bidirectional ports
wire   <signal_names>;         // net declarations
reg    <signal_names>;         // register declarations
parameter <name> = <value>;    // constants
// — Body / Statements —
initial statement;              // executes once at time=0
always statement;               // executes repeatedly
// module instantiation
// continuous assignment (assign)
endmodule
```

Important Syntactic Rules:

- Verilog is case-sensitive: module ≠ Module ≠ MODULE
- Module name must start with an alphabetical letter; it can include underscores (_)
- Semicolons (;) are used as statement terminators / line separators
- Order of input/output declarations is irrelevant to functionality
- "=" is the assignment operator for procedural assignments
- "^" means XOR, "&" means AND, "|" means OR, "~" means NOT in Verilog
- "//" begins a single-line comment; /* ... */ is used for multi-line comments
- Every module begins with "module" and ends with "endmodule"

3.2 Verilog Ports

Ports define the interface of a module — i.e., how signals enter and leave a module. There are three types of ports:

Port Type	Direction	Usage	Assignment Rule
input	Signal flows INTO the module	Used to receive data from external circuits	Can only appear on the RIGHT-HAND SIDE (RHS) of assignments — read only
output	Signal flows OUT of the module	Used to send computed results outside the module	Can appear on either LHS or RHS of assignments
inout	Bidirectional signal	Used for bidirectional buses (e.g., data bus)	Can be used for both reading and writing — represents tri-state bus

3.3 Half Adder Example

A half adder computes the sum and carry of two single-bit binary numbers.

Truth Table:

A	B	Sum (A XOR B)	Carry (A AND B)
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Verilog Code – Half Adder:

```
module halfadder (a, b, sum, carry);  
    input a;          // input port a  
    input b;          // input port b  
    output sum;        // output port sum  
    output carry;      // output port carry  
  
    assign sum  = a ^ b; // XOR for Sum  
    assign carry = a & b; // AND for Carry  
  
endmodule
```

Here, two continuous assignment statements describe the combinational logic of a half adder. The "assign" keyword makes these **continuously active** — any change on a or b immediately updates sum and carry.

 **NOTE:** The "module" keyword starts the definition and "endmodule" closes it. All port declarations and logic must lie between these two keywords.

4. Verilog Operators

Verilog supports an extensive set of operators that perform a wide variety of functions on signals and variables. Operators are classified into four main categories:

9. Logical Operators – AND, OR, NAND, NOR, XOR, XNOR, NOT
10. Relational Operators – compare two values ($=$, $\neq$, $<$, $\leq$, $>$, $\geq$)
11. Arithmetic Operators – addition, subtraction, multiplication, division, modulus, exponent
12. Shift Operators – move bits left or right

4.1 Logical Operators

Logical operators perform bit-wise and boolean operations on operands. They are further classified into three sub-types:

4.1.1 Bitwise Operators

Bitwise operators operate on the corresponding bits of two operands and produce a result of the same width. They are similar to VHDL logical operators.

Verilog Operator	Operation	Operand Type	Result Type	Example
&	Bitwise AND	Bit	Bit	$4'b1011 \& 4'b1010 = 4'b1010$
	Bitwise OR	Bit	Bit	$4'b1011 4'b1010 = 4'b1011$
^	Bitwise XOR	Bit	Bit	$4'b1011 \wedge 4'b1010 = 4'b0001$
~	Bitwise NOT	Bit	Bit	$\sim 4'b1011 = 4'b0100$
~(&)	Bitwise NAND	Bit	Bit	NAND of all bits
~()	Bitwise NOR	Bit	Bit	NOR of all bits
~^	Bitwise XNOR	Bit	Bit	XNOR of corresponding bits

Example: $Z = x \& y$, if $x=1011$ and $y=1010$ (4-bit signals), then $Z = 1010$ (bitwise AND).

4.1.2 Boolean Logical Operators

Boolean operators treat each operand as a single truth value (true if non-zero, false if zero). The result is a 1-bit Boolean (0 or 1).

Operator	Operation	Operand s	Result	Example
&&	Logical AND	Two	0 or 1	$x=1011, y=0001 \rightarrow Z=1$ (both non-zero, so $\text{TRUE} \&\& \text{TRUE} = 1$)
	Logical OR	Two	0 or 1	$x=0000, y=0001 \rightarrow Z=1$
!	Logical NOT	One	0 or 1	$!1111 \rightarrow 0$ (non-zero $\rightarrow$ false $\rightarrow 0$)

Example: $z = x \&\& y$; if $x=1011$ and $y=0001$, then $Z=1$. If $x=1010$ and $y=0101$, then $Z=0$ (wait — both are non-zero, so $Z=1$ actually). $z! = x$ means logical NOT of x ; if $x=1111$ then $z=0$.

4.1.3 Reduction Operators

Reduction operators take a single multi-bit operand and reduce it to a 1-bit Boolean result by applying the operation across all bits of the operand.

Operator	Operation	Operands	Example (x=4'b1010)	Result
&	Reduction AND	One	&x = 1&0&1&0	0
	Reduction OR	One	x = 1 0 1 0	1
~(&)	Reduction NAND	One	~(&x) = ~(0)	1
~()	Reduction NOR	One	~(x) = ~(1)	0
^	Reduction XOR	One	^x = 1^0^1^0	0
~(^)	Reduction XNOR	One	~(^x) = ~(0)	1
!	Negation	One	!x = !(1010)	0 (since x is non-zero)

 **NOTE:** Reduction operators are extremely useful for checking parity, detecting all-zeros, all-ones conditions in a bus or vector.

4.2 Relational Operators

Relational operators compare the values of two operands. The result is:

- 0 (false) – if the condition is NOT met
- 1 (true) – if the condition IS met
- X (unknown) – if any operand contains don't-care (X) or high-impedance (Z) bits

Operator	Description	Result Type	Notes
==	Equality	0, 1, X	Returns X if any bit is X or Z
!=	Inequality	0, 1, X	Returns X if any bit is X or Z
===	Case Equality (Inclusive)	0, 1 only	Compares X and Z bits exactly — no unknown result
!==	Case Inequality (Inclusive)	0, 1 only	Exact bit comparison including X and Z
<	Less than	0, 1, X	—
<=	Less than or equal to	0, 1, X	—
>	Greater than	0, 1, X	—
>=	Greater than or equal to	0, 1, X	—

Key Note on === vs ==: The == operator returns X when operands contain X or Z bits. The === operator (case equality) compares all four logic values (0, 1, X, Z) exactly and always returns 0 or 1 — very useful in simulation test benches.

Example: If A=4'b101X and B=4'b101X, then (A==B) returns X, but (A===B) returns 1.

4.3 Arithmetic Operators

Arithmetic operators perform standard mathematical operations on numeric operands. Verilog is not an extensive type-oriented language for arithmetic, but it supports all basic operations.

Operator	Operation	Example	A/B Type	Y (Result) Type
+	Addition	A+B	Numeric	Numeric
-	Subtraction	A-B	Numeric	Numeric
*	Multiplication	A*B	Numeric	Numeric
/	Division	A/B	Numeric	Numeric (integer division)
%	Modulus	A%B	Integer (not real)	Integer (not real)
**	Exponentiation	A**B	Numeric	Numeric
{,}	Concatenation	{A,B}	Numeric/Array	Same type as A

Example: $y = (A * B)$ calculates the product of A and B.

Concatenation Example: $\{4'b1010, 4'b0101\} = 8'b10100101$ — two 4-bit values concatenated into one 8-bit value.

4.4 Shift and Rotate Operators

Shift operators move the bits of a vector left or right by a specified number of positions. These are unary operators (operate on a single operand).

Key Principle: A left shift by 1 is equivalent to multiplication by 2. A right shift by 1 is equivalent to division by 2.

Operation	Description	Before Shift (A=4'b1110)	After Shift	Vacated Bits
A << 1	Shift one position LEFT (logical)	1110	1 1 0 X (= 110X)	Filled with X
A << 2	Shift two positions LEFT (logical)	1110	1 0 X X (= 10XX)	Filled with X
A >> 1	Shift one position RIGHT (logical)	1110	X 1 1 1 (= X111)	Filled with X
A >> 2	Shift two positions RIGHT (logical)	1110	X X 1 1 (= XX11)	Filled with X

 **NOTE:** In synthesis, shift by a constant is synthesized as wiring (no logic gates needed). Shift by a variable requires a barrel shifter circuit.

5. Verilog Data Types

Data types in Verilog define the type of data a signal, variable, or element can hold. Different types are needed to accurately model the various elements of hardware.

There are seven categories of Verilog data types:

13. Nets
14. Registers
15. Vectors
16. Integer
17. Real
18. Parameters
19. Array

5.1 Nets (wire)

Nets represent physical wires (connections) in a circuit. They are the most commonly used data type in Verilog. Nets are declared using the keyword **"wire"**.

Key Properties of Nets:

- Net values are driven continuously by the circuits that are driving them.
- A wire represents a physical wire that connects gates or modules.
- The value of a wire can be READ but NOT ASSIGNED inside a function or procedural block.
- If no driver is connected, a wire takes the value Z (high impedance).

Four Logic Values Supported by Nets:

Value	Net Definition	Register (reg) Definition
0	Logic 0 (false)	Logic 0
1	Logic 1 (true)	Logic 1
X	Unknown logic value	Unknown
Z	High impedance (tri-state)	High impedance

Examples:

```

wire sum;           // declares a net named sum
wire s1 = 1'b0;     // declares net s1, initialised to 1-bit 0
wire [7:0] databus; // declares an 8-bit wide net called databus

```

5.2 Registers (reg)

Registers are data storage elements that hold their value until explicitly updated. They do NOT necessarily map to physical flip-flops in hardware — they simply retain their last assigned value in simulation.

Key Properties of Registers:

- Declared using the keyword **"reg"**
- Values are assigned using procedural statements inside **"always"** or **"initial"** blocks
- Registers hold their value between assignments — unlike wires which need continuous driving
- Supports the same four logic values as nets: 0, 1, X, Z

```

reg sum_total;      // declares a 1-bit register named sum_total
reg [7:0] counter;  // declares an 8-bit register
reg [3:0] nibble = 4'b0000; // register with initial value

```

 **NOTE:** In synthesis: if a reg is assigned in an always @(posedge clk) block, it synthesizes to a D flip-flop. If assigned in always @(*), it synthesizes to combinational logic.

5.3 Vectors

Vectors are multi-bit signals. Both "reg" and "wire" can be declared as vectors. They are declared by specifying a range in square brackets **[MSB:LSB]** before the signal name.

- [MSB:LSB] → Big-endian (most significant bit is the higher index): [3:0] = 4-bit vector
- [LSB:MSB] → Little-endian (least significant bit first): [0:3] = 4-bit vector (same width, different ordering)

Examples:

```
wire [3:0] a = 4'b1010;    // 4-bit net a with value 1010
reg  [7:0] total = 8'd12;  // 8-bit reg with decimal value 12
wire [15:0] address_bus;  // 16-bit address bus
// Accessing individual bits:
wire msb = a[3];          // MSB of a
wire lsb = a[0];          // LSB of a
wire [1:0] slice = a[3:2]; // 2-bit slice of a
```

5.4 Integer, Real, Parameters, and Arrays

5.4.1 Integer

Integers are general-purpose signed variables used mainly for loop indices, parameters, and constants in synthesis. They are declared using the keyword **"integer"**.

```
integer no_bits;          // declares an integer variable
integer i, j;             // multiple integers
integer count = 0;        // integer with initial value
```

Integers are 32 bits wide by default and are signed (can hold negative values using 2's complement).

5.4.2 Real

Real numbers (floating-point) are declared with the keyword **"real"**. They are primarily used in simulation for more precise arithmetic. They are NOT synthesisable.

```
real weight;              // declares a real-type register
real pi = 3.14159;        // real with initial value
```

Examples of real values: 2.4, 56.3, 5e12 (5×10^{12}).

5.4.3 Parameters

Parameters represent named constants — their value is fixed at compile time and cannot change during simulation. They are declared using the keyword **"parameter"** and are used to make modules more flexible and reusable.

```
parameter N = 3;          // integer parameter N = 3
parameter WIDTH = 8;

// Example usage in module:
module comp_genr (x, y, xgty, xlty, xeqy);
  parameter N = 3;
  input  [N:0] x, y;
  output xgty, xlty, xeqy;
  wire  [N:0] sum, xb;
endmodule
```

By changing the parameter N, the same module can be used for different word widths without rewriting code.

5.4.4 Arrays

Verilog does not have a dedicated "array" keyword. Arrays are created using reg or integer declarations followed by a range specifier **[start:end]** after the signal name.

```
parameter N = 4;
parameter M = 3;
reg signed [M:0] carry [0:N]; // Array of 5 signed 4-bit registers
reg [M:0] b [0:N];           // Array of 5 unsigned 4-bit registers
integer sum [0:N];           // Array of 5 integers
```

In the above: array sum has 5 elements (index 0 to 4), each an integer. Array carry has 5 elements, each 4 bits wide in 2s complement form.

6. Styles (Types) of Descriptions

Verilog supports multiple styles (levels of abstraction) for describing hardware. Each style offers a different perspective and level of detail.

6.1 Behavioral Description

Behavioral description models the system in terms of **HOW THE OUTPUTS BEHAVE WITH INPUTS** — i.e., it describes the algorithm or function, not the physical implementation.

Identification: A Verilog module that uses the keyword "**always**" is behaviorally described.

Key Characteristics:

- Uses "always" or "initial" procedural blocks
- Describes how the system behaves (input-output relationship)
- High-level abstraction — no explicit gate-level structure
- Pure behavioral: contains ONLY "always"/"initial" — no structural or dataflow elements
- Sequential statements inside always/initial blocks execute sequentially (unlike concurrent assignments)

Example – Behavioral D Flip-Flop:

```
module dff (d, clk, q);
    input  d, clk;
    output q;
    reg q;
    always @(posedge clk) // triggered on rising edge
        q <= d;           // non-blocking assignment
endmodule
```

6.2 Structural Description

Structural description models the system as an interconnection of **components or gates**. It describes the physical structure of the circuit rather than its behaviour.

Identification: Uses gate primitives like "and", "or", "not" or module instantiation (component instantiation).

Key Characteristics:

- Uses built-in gate primitives: and, or, not, nand, nor, xor, xnor, buf
- Or uses user-defined modules instantiated as sub-components
- Pure structural: only gates and module instances — no "always" or "assign"
- Directly maps to physical gate-level circuit

Example – Structural Half Adder:

```
module halfadder_struct (a, b, sum, carry);
    input  a, b;
    output sum, carry;
    xor g1 (sum, a, b); // gate instantiation
    and g2 (carry, a, b);
endmodule
```

6.3 Dataflow Description

Dataflow description describes how signals flow from inputs to outputs. It uses continuous assignment statements (assign) to express the Boolean functions of outputs. The execution of these statements is controlled by events on the RHS signals.

Identification: Uses the "assign" keyword with Boolean expressions.

```
module halfadder_df (a, b, sum, carry);  
    input  a, b;  
    output sum, carry;  
    assign sum  = a ^ b;  
    assign carry = a & b;  
endmodule
```

6.4 Switch Level Description

Switch level is the lowest level of hardware abstraction. The system is described using MOS transistors (switches).

Identification: Uses Verilog keywords such as: **nmos**, **pmos**, **cmos**, **tran**, **tranif0**, **tranif1**.

- nmos – n-channel MOS transistor
- pmos – p-channel MOS transistor
- cmos – complementary MOS switch
- tran – bidirectional switch

VHDL does not have built-in switch level primitives; packages are required for such descriptions. This style is not commonly used in modern synthesis flows.

6.5 Mixed-Type Description

A mixed-type description uses more than one style in a single module. For example, some parts of the module may use dataflow (assign) while other parts use behavioral (always) style.

Example use case: A module may describe the control logic behaviorally and the datapath using dataflow statements. This is very practical for real-world designs.

6.6 Mixed-Language Description

Mixed-language description allows the integration of VHDL and Verilog in the same design. A module written in Verilog can instantiate (invoke) an entity written in VHDL and vice versa.

Modern EDA (Electronic Design Automation) tools such as Modelsim, Cadence Incisive, and Synopsys VCS support mixed-language simulation, making this a practical option for large design teams that may use both languages.

7. Verilog Dataflow Description (Detailed)

7.1 Highlights of Dataflow Description

Dataflow description is a type of hardware description which shows how signals flow from system inputs to system outputs. It is the preferred style for describing combinational logic circuits.

Key Highlights:

- Uses signal assignment statements that are executed concurrently (simultaneously).
- An assignment statement executes when an EVENT occurs on any signal on the Right Hand Side (RHS).
- The description is typically done by writing the Boolean functions of the outputs.
- No explicit structure (gates, flip-flops) is described — only the logical function.
- All combinational circuits (adders, muxes, encoders, comparators) are naturally described using dataflow style.

7.2 Concurrent vs Sequential Program Execution

In HDL, there are two fundamental methods of program execution:

7.2.1 Concurrent Execution

In concurrent execution, all statements in the program execute simultaneously, irrespective of their order. All combinational circuits require this style.

Example: A two-input gate network with outputs $Y1 = A \& B$ and $Y2 = A \wedge B$. Both outputs are evaluated simultaneously whenever A or B changes — this is concurrent execution.

Trigger: Any event (change) on A or B causes BOTH Y1 and Y2 to update simultaneously, regardless of statement order.

- All "assign" statements in Verilog are concurrent — they run in parallel.
- The order of assign statements in the source code does NOT affect simulation results.

7.2.2 Sequential Execution

In sequential execution, all statements are executed one after another in the order of their appearance, like a conventional software program.

Example: A D flip-flop — the data at input D is transferred to output Q only after the rising or falling edge of the clock. The clock triggers execution in a sequential manner.

- All sequential circuits (flip-flops, counters, shift registers, FSMs) require sequential execution.
- Implemented using "always @(posedge clk)" or "always @(negedge clk)" blocks.

Aspect	Concurrent Execution	Sequential Execution
Keyword	assign (continuous assignment)	always / initial (procedural)
Execution order	All statements execute simultaneously	Statements execute one-by-one, top to bottom
Trigger	Event on any RHS signal	Clock edge or sensitivity list event
Circuit type	Combinational logic	Sequential logic (flip-flops, FSM)
Example	assign Y = A & B	always @(posedge clk) Q <= D;

7.3 Structure of Dataflow Description

A dataflow model specifies the functionality of the system **without explicitly specifying its structure**. It describes how signals flow from inputs to outputs using Boolean logic functions.

EVENT: An event is a change in the value of a signal — such as a transition from logic 0 to logic 1 (rising edge) or from 1 to 0 (falling edge). Events trigger the execution of signal assignment statements.

Dataflow description is modelled using:

- Continuous signal assignment statements in Verilog (using the assign keyword)
- Concurrent signal assignment statements in VHDL (using the <= operator)

Example – Dataflow System Module:

```
module system (I1, I2, O1, O2);
  input  I1, I2;
  output O1, O2;
  assign O1 = I1 & I2; // statement 1 (concurrent)
  assign O2 = I1 ^ I2; // statement 2 (concurrent)
endmodule
```

In this module, I1 and I2 are inputs. O1 and O2 are outputs. Statements 1 and 2 are both active simultaneously — if I1 or I2 changes, both O1 and O2 are updated in the same simulation step.

7.4 Signal Declaration

In a Verilog dataflow module, signals are declared as follows:

- Input and output signals (ports) are declared in the module header.
- Intermediate signals (internal signals that are neither inputs nor outputs) are declared using the keyword "wire" in Verilog.
- When the value of a signal needs to be stored (retained between clock edges), it is declared as "reg".

```
wire s1, s2;           // intermediate net signals
wire [3:0] result;     // 4-bit intermediate net
reg  stored_val;       // signal that stores a value
```

Without proper signal declarations, the Verilog simulator will throw undeclared identifier errors. All signals used in RHS expressions must be declared.

7.5 Signal Assignment Statements

A signal assignment statement assigns a value to a signal based on the evaluation of an expression. In Verilog dataflow modeling:

- The LHS (Left Hand Side) of the assignment must be a declared signal or net.
- The RHS (Right Hand Side) can be a signal, a variable, a constant, or a complex expression.
- The assignment operator in Verilog for continuous assignments is the keyword "assign" combined with "=".

Execution of a signal assignment has TWO phases:

20. CALCULATION PHASE: When an event occurs on any RHS signal, the RHS expression is evaluated using the current values of all signals. The result is computed but NOT yet assigned.
21. ASSIGNMENT PHASE: After a delta (Δ) delay (or explicit delay), the computed result is assigned to the LHS signal. The Δ delay is an infinitesimally small simulation time increment.

Example: assign O1 = I1 & I2;

If an event at time T0 changes I1 from 0→1 and I2 from 0→1:

- Calculation: 1 & 1 = 1 (computed at T0, not yet assigned)
- Assignment: O1 = 1 (assigned at T0 + Δ)

7.6 Continuous Assignment Statements

A continuous assignment statement is the most basic statement in Verilog dataflow modeling. It is used to drive a value onto a net continuously and is triggered by any event on the RHS operands.

Syntax:

```
assign LHS_target = RHS_expression;
```

Example 1:

```
wire [3:0] Z, preset, clr;
assign Z = preset & clr; // Z is updated whenever preset or clr changes
```

Example 2 – Full Adder:

```
module full_adder (a, b, cin, sum, cout);
    input  a, b, cin;
    output sum, cout;
    assign sum  = a ^ b ^ cin;
    assign cout = (a & b) | (b & cin) | (a & cin);
endmodule
```

Properties of Continuous Assignments:

- Always active — executes continuously throughout simulation
- The LHS must be a net (wire), NOT a register (reg)
- Executes whenever any operand on the RHS changes
- Multiple continuous assignments to the same wire create contention (avoid this)

7.7 Concurrent Signal Assignment Statements (VHDL Context)

In VHDL, the equivalent of Verilog's continuous assignment is the concurrent signal assignment. It uses the <= operator.

VHDL Syntax:

```
LHS_target <= RHS_expression;
Example:  C <= A and B;
```

The value computed by RHS_expression (A and B) is assigned to the LHS_target C. The <= operator is called the signal assignment operator in VHDL.

Behavior in Verilog dataflow: All assign statements in a Verilog module are concurrent. If events occur on multiple assignment statements simultaneously, ALL of them execute at the same time regardless of their order in the source code.

7.8 Constant Declaration and Assignment

Constants hold a fixed value that does not change during simulation within the scope where they are declared.

In Verilog, constants are declared by type:

```
time period;           // time-type constant
integer count_max;     // integer-type constant
```

Assignment:

```
period      = 100;      // 100 simulation time units (no explicit units in Verilog)
count_max = 255;
```

Combined declaration and assignment:

```
parameter time period = 100; // Verilog
```

Note: In Verilog, there are no explicit units of time. The value 100 means 100 simulation screen time units. The 'timescale compiler directive can set the time unit and precision.

7.9 Conditional Signal Assignment Statement

The conditional signal assignment selects one out of multiple values for a target signal based on specified Boolean conditions. This is a VHDL construct (when-else), but Verilog achieves the same effect using the conditional operator (?:) or the assign-with-ternary.

VHDL Syntax (for understanding):

```
target_signal <= expression1 when boolean_condition1 else
                    expression2 when boolean_condition2 else
                    expression_n;
```

Execution Mechanism:

22. An event occurs on any signal in the condition or expression.
23. boolean_condition1 is evaluated first (highest priority).
24. If TRUE → expression1 is assigned to target.
25. If FALSE → check condition2, and so on.
26. If no condition is TRUE → expression_n (default) is assigned.

Special Feature: Although when-else is a concurrent statement, the conditions are evaluated sequentially — so condition1 has the highest priority. This is used to implement priority encoders!

VHDL Example – 4:1 MUX using when-else:

```
entity MUX is
    port (A, B, C, D : in std_logic;
          SEL         : in std_logic_vector(1 downto 0);
          Z           : out std_logic);
end MUX;
architecture MUX41 of MUX is
begin
    Z <= A when SEL = "00" else
        B when SEL = "01" else
        C when SEL = "10" else
        D;
end MUX41;
```

Verilog Equivalent (Conditional Operator):

```
assign Z = (SEL==2'b00) ? A :
           (SEL==2'b01) ? B :
           (SEL==2'b10) ? C : D;
```

7.10 Selected Signal Assignment Statement

The selected signal assignment (with-select in VHDL) provides selective, non-priority signal assignment. Unlike when-else which checks conditions sequentially, selected assignment evaluates all choices simultaneously — making all choices equal priority.

VHDL Syntax:

```
with choice_expression select
    target_name <= expression1 when choices1,
                  expression2 when choices2,
                  expression_n when choices_n;
```

Execution Mechanism:

27. An event occurs on any signal in choice_expression or any of the expressions.
28. The choice_expression is evaluated first.
29. The computed value is compared with ALL choices simultaneously.
30. The expression whose choice matches is assigned to target_name.

Rules for Choices:

- All possible values of choice_expression must be covered exactly once.
- Values not covered explicitly must use an "others" clause.
- Choices must be static expressions (constants).
- Choices must be mutually exclusive — no value can appear in two choice lists.

Important: The choice_expression MUST contain at least one signal (not just constants) because an EVENT can only occur on a signal. For example, if choice_expression is "X+Y", then at least one of X or Y must be a signal.

Verilog with case statement (equivalent):

```
always @(*) begin
  case (SEL)
    2'b00: Z = A;
    2'b01: Z = B;
    2'b10: Z = C;
    2'b11: Z = D;
    default: Z = 1'bX;
  endcase
end
```

7.11 Assigning Delay Time to Signal Assignment

In Verilog, propagation delay can be added to a continuous assignment using the "#" symbol followed by the delay value in simulation time units.

Syntax:

```
assign #<delay_time> LHS = RHS_expression;
```

Example:

```
assign #10 s1 = sel & b; // 10 simulation time unit delay
assign #5 sum = a ^ b; // 5 time unit gate delay
```

Note: In Verilog, you cannot specify explicit units of delay. The number refers to simulation screen time units. Use `timescale to set the unit and precision.

Example 2.1 – Dataflow Half Adder:

```
module halfadder_df (a, b, sum, carry);
  input a, b;
  output sum, carry;
  assign #2 sum = a ^ b; // 2 unit delay on sum
  assign #2 carry = a & b; // 2 unit delay on carry
endmodule
```

Example 2.2 – 2×1 MUX with Active Low Enable:

```
module mux2x1_en (A, B, SEL, EN_n, Y);
  input A, B, SEL, EN_n; // EN_n = active-low enable
  output Y;
  wire sel_a, sel_b;
```

```
assign sel_a = ~SEL & ~EN_n;  
assign sel_b =  SEL & ~EN_n;  
assign Y      = (A & sel_a) | (B & sel_b);  
endmodule
```

 **NOTE:** When EN_n=1 (disabled), both sel_a and sel_b are 0, so Y=0 regardless of A, B, or SEL.

8. Quick Revision – All Topics at a Glance

 This section provides a concise exam-oriented recap of ALL topics in Module 4.

8.1 HDL Basics

Concept	Key Point
HDL Definition	Computer-aided design (CAD) tool for digital design, simulation, and synthesis
Two HDLs	VHDL (verbose, strongly typed) and Verilog (C-like, widely used in industry)
Need for HDL	Discrete ICs insufficient for complex systems; HDL enables ASIC/FPGA design
Verilog Origin	Gateway Design Automation, 1983; IEEE Standard 1364-1995
Verilog Basis	Based on C programming language syntax

8.2 Module Structure

Term	Meaning
module ... endmodule	Defines a Verilog module block
Port list	Lists all input/output/inout signals in module declaration
input	Read-only port — can only appear on RHS of assignment
output	Driven port — can appear on LHS or RHS of assignment
inout	Bidirectional port — used for shared buses
assign	Continuous assignment keyword (dataflow)
Case sensitivity	Verilog is CASE SENSITIVE — module ≠ Module

8.3 Operators Summary

Category	Operators	Key Note
Bitwise Logical	& ^ ~ ^~ ^~() ~(&)	Operate bit-by-bit on two operands
Boolean Logical	&& !	Result is always 0 or 1 (true/false)
Reduction	& ^ ~ ^~ ^~() ~(&) !	Operates on ONE operand, reduces to 1 bit
Relational	== != === !== < <= > >=	=== includes X/Z comparison (no unknown result)
Arithmetic	+ - * / % ** {,}	% not for real; {,} is concatenation
Shift	<< >>	<< = multiply by 2; >> = divide by 2

8.4 Data Types Summary

Data Type	Keyword	Key Points
Net	wire	Physical wire; driven continuously; value = Z if undriven; 4 logic values (0,1,X,Z)
Register	reg	Data storage; holds value until updated; used in always/initial blocks
Vector	[MSB:LSB]	Multi-bit wire or reg; accessed by range e.g. a[3:0]
Integer	integer	32-bit signed variable; used for loops, counters, parameters
Real	real	Floating-point; simulation only, NOT synthesisable
Parameter	parameter	Named constant; makes modules reusable and configurable
Array	reg/integer [range]	No "array" keyword; use reg or integer with size range

8.5 Styles of Description Summary

Style	Identification Keyword	Circuit Type	Abstraction Level
Behavioral	always, initial	Sequential (flip-flops, FSM)	Highest — algorithm level
Structural	and, or, not, module inst.	Both combinational & sequential	Medium — gate/component level
Dataflow	assign	Combinational (adder, mux, encoder)	Medium — Boolean equation level
Switch Level	nmos, pmos, cmos, tran	Transistor-level circuits	Lowest — transistor level
Mixed Type	Combination of above	Any	Multiple levels
Mixed Language	VHDL + Verilog	Any	Project-level integration

8.6 Dataflow Description Key Concepts

Concept	Key Definition
Event	A change in signal value (0→1 or 1→0) that triggers assignment execution
Concurrent execution	All assign statements execute simultaneously — order does NOT matter
Sequential execution	Statements in always/initial execute top-to-bottom in order
Continuous assignment	assign Z = A & B; — LHS updated whenever A or B changes
Signal declaration	wire = intermediate nets; reg = stored values
Delay assignment	assign #10 Y = A & B; — 10 time unit propagation delay
Conditional assignment	when-else (VHDL) or ?: (Verilog) — priority-based selection
Selected assignment	with-select (VHDL) or case (Verilog) — simultaneous comparison, equal priority
Delta delay (Δ)	Default infinitesimal delay between calculation and assignment phases

8.7 Must-Remember Code Snippets

1. Half Adder (Dataflow):

```
assign sum = a ^ b; assign carry = a & b;
```

2. Full Adder (Dataflow):

```
assign sum = a ^ b ^ cin; assign cout = (a&b) | (b&cin) | (a&cin);
```

3. 2:1 MUX (Conditional Operator):

```
assign Y = SEL ? B : A;
```

4. 4:1 MUX (Nested Ternary):

```
assign Y = (S==2'b00)?A:(S==2'b01)?B:(S==2'b10)?C:D;
```

5. D Flip-Flop (Behavioral):

```
always @(posedge clk) q <= d;
```

6. 4-bit AND reduction:

```
wire result = &a; // result = a[3] & a[2] & a[1] & a[0]
```

8.8 Common Exam Questions

31. What is HDL? Explain the need for HDL. (4–6 marks)
32. Explain the structure of a Verilog module with syntax and example. (6–8 marks)
33. Classify and explain Verilog operators with examples and tables. (8–10 marks)
34. Explain Verilog data types with examples. (8–10 marks)
35. Differentiate between behavioral, structural, and dataflow descriptions with examples. (6–8 marks)
36. Explain the difference between concurrent and sequential execution. (4–6 marks)
37. Write a Verilog dataflow module for a half adder. (4 marks)
38. Explain continuous signal assignment and concurrent signal assignment. (6 marks)
39. What is a conditional signal assignment? Explain with priority encoder example. (6 marks)
40. Explain selected signal assignment. What are the rules for choices? (6 marks)

END OF MODULE 4 – ALL THE BEST FOR YOUR EXAMS! 🎓

vtu-easy.com